

Two Regions, One NATS Supercluster

Ten patterns measured on a six-server lab — three in South Africa, three in Australia, joined by a gateway. Each card states the mechanism, what it costs, and the number the lab actually printed. Nothing here is from a blog post.

THE QUESTION EVERY CARD SERVES

A truck drives **12.5 km, once**, in South Africa. Two regions are listening. What does the odometer say afterwards — and what happens to both regions when the link between them dies?

THE RULE THAT ORDERS THE DECK

A gateway buys **reach** and sells you a **shared namespace and a shared failure domain**. Only the **account** is a wall. Everything else on these pages — subjects, placement, domains — is a convention, a location, or a name.

THE TEN PATTERNS

- 01 · One supercluster takes one JetStream domain
- 02 · The stream-name collision — error 10058
- 03 · Placement — where a stream lives
- 04 · A region token in the subject — the convention
- 05 · One account per region — the wall
- 06 · Replicas and quorum — 1, 3, 5
- 07 · What replication costs — the acknowledgement
- 08 · The shared failure domain — a WAN cut
- 09 · Escaping the shared failure domain
- 10 · Measuring without fooling yourself

01

NAMESPACE

One supercluster takes one JetStream domain

A gateway joins two clusters into one **supercluster**. A supercluster is one JetStream system, and a JetStream system has exactly one **domain** name. Every server carries the same one. Here it is **lb**.

DECISION

Can a per-region **domain** (**za**, **au**) be used to keep two regions apart? **No. It is not a legal configuration**, and the servers do not tell you.

MECHANISM



- › A domain names a **JetStream system**, not a region and not a tenant. The same name is required on every server of a cluster **and** of a supercluster.
- › The name may only change across a **leaf-node** link. A gateway is not a leaf link.
- › Setting **domain** also suppresses JetStream traffic on the system account — which is what silently breaks a supercluster that uses two names.

WHAT IT DOES GIVE YOU

- › An **address**. `$JS.lb.API.>` lets a leaf node reach a named JetStream from outside it.
- › Clean addressing when you really do run separate JetStream systems joined by leaf nodes — see Card 09.

WHAT IT DOES NOT GIVE YOU

- › **Not a tenant boundary.** That is the account — Card 05.
- › **Not a location.** That is placement — Card 03.
- › **Not availability.** A domain does not change the meta group — Card 08.
- › **No error if you get it wrong.** Two domains in one supercluster produce silence, not a message.

MEASURED, THEN REVERSED — 2026-09-09

- › This repo shipped **za** / **au** / **hub** domains, on the belief that a domain was a per-cluster boundary. It broke JetStream instead. The correction is dated in [docs/Multi-Region-Plan.md](#) at D9 and D11.
- › The lab now runs **one domain**, **lb**, on all six servers, and separates regions with accounts and placement.

VERDICT

Treat **domain** as an **address for leaf nodes**, nothing else. If you are reaching for it to split two clusters that share a gateway, you are solving the wrong problem with the wrong tool.

boundary strength: none

02

NAMESPACE

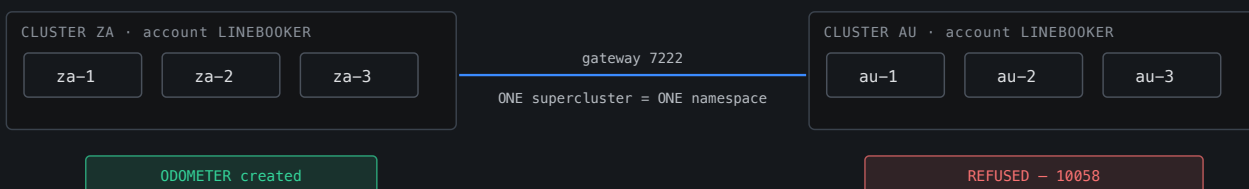
The stream-name collision — error 10058

Inside one account, a stream name is unique across the **whole supercluster**. South Africa creates `ODOMETER`. Australia asks for `ODOMETER` and is refused. There is no second stream, and no warning that there is not.

DECISION

Two regions both want a stream with the same, obvious, domain-correct name. Which axis do you spend to get it — the **subject** (Card 04), or the **account** (Card 05)?

MECHANISM — MEASURED BY `LAB/00-THE-PROBLEM.SH`



- › The exact text: `stream name already in use with a different configuration (10058)`.
- › Worse than the error: `stream info` from AU then **succeeds** and returns ZA's stream — same `cluster`, same `created` timestamp. AU has a working handle to somebody else's data.
- › One `purge` in ZA empties "both". They were always one object.

WHY THIS IS THE DANGEROUS ONE

- › **The failure is at create time, once, in a script nobody reads twice.** After that everything looks healthy.
- › Each region still runs its own writer and its own projector. They now share one log, so one business event is applied **once per region**.
- › A message count cannot show this. Only a **business total** can — 25 km for a 12.5 km trip. See Card 05.
- › If AU goes off the air, AU loses its data — because AU never had any.

WHAT DOES NOT FIX IT

- › **A domain.** Card 01.
- › **Placement.** `--cluster au` chooses where a stream lives. It cannot make a second stream with a name the account has already used.
- › **Bigger clusters, more replicas, a hub.** None of these touch the namespace.

VERDICT

A shared account across a gateway is a **shared name space, a shared log and a shared bug**. Discover it on day one with a deliberate collision test, not in month six with a wrong invoice.

detection: create-time only

03

LOCATION

Placement — where a stream physically lives

In a supercluster, JetStream will otherwise choose for you. `--cluster za` pins the stream and every one of its replicas to the South African servers. It is the right tool for *where*, and it is not a boundary.

DECISION

Where must this data **sit** — for latency, for data residency, for the blast radius of one region failing?

MECHANISM



- › **A stream's replicas never span clusters.** Placement is not "prefer" — it is the fence the Raft group is built inside. Asking for more replicas than one cluster holds fails with `no suitable peers for placement (10005)`, even when the supercluster has plenty of servers.
- › Omitting `--cluster` does not spread a stream. It lets the meta leader pick **one** cluster, possibly the far one.
- › Check it, do not assume it: `nats stream info <name> --json | jq .cluster.name`. The lab's `lab/streams.sh` prints this for all six contexts.

PROS

- › **Real and physical.** Data residency and read latency are decided here, not by naming.
- › Cheap — one flag, per stream, no trust-chain change.
- › Makes a region's failure legible: you know exactly which streams went with it.

CONS

- › **Not a security or correctness boundary.** Anyone in the account can read a ZA-placed stream from AU.
- › Does not solve the name collision — Card 02.
- › Silent by omission: forget the flag and placement happens anyway, somewhere.
- › Caps your replica count at the size of one cluster.

SAY IT ONCE, CLEARLY

- › **Domain = which JetStream system. Placement = which cluster. Account = who may see it.** Three different words, three different jobs, and only the third one is a wall.

VERDICT

Set `--cluster` explicitly on every stream in a supercluster. It costs one flag and it removes an entire class of "why is our data in Sydney" question. boundary strength: none · location: exact

04

SEPARATION, BY CONVENTION

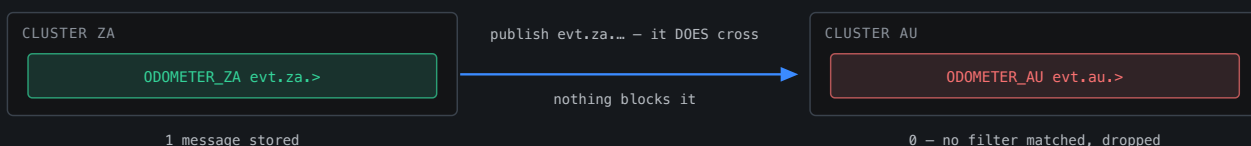
A region token in the subject

Keep one account. Put the region in the subject, so the two streams no longer overlap: ZA filters `evt.za.>`, AU filters `evt.au.>`. It works. It is a naming convention, not a boundary.

DECISION

Are you willing to pay for separation in **subjects and stream names** — every publisher, every consumer, every filter — to avoid changing the trust chain?

MECHANISM — MEASURED BY `LAB/01-OPTION-1-SUBJECTS.SH`



- › **The region token is paid TWICE.** Once in the subject, and once in the stream **name** — because Card 02 still applies inside one account.
- › The message **did** cross the gateway. Nothing blocked it. AU simply had no filter that matched and threw it away.
- › What physically held the two streams apart was `--cluster`, not the subject.

PROS

- › **No trust-chain change.** No new accounts, no new credentials, no re-issue.
- › Stops the double capture today, with a filter edit.
- › Easy to reason about in a small system with few publishers.

CONS

- › **A typo restores the bug, silently.** One publisher with the wrong prefix and the double capture is back with no error anywhere.
- › Spends two of this repo's settled design rules: region is **not** a subject token, and streams are named for the domain, not the deployment.
- › Touches every publisher, consumer and filter — a wide, boring migration.
- › Still leaves **two write sides in one account**, with no story for merging them.

VERDICT

It stops the double capture and fixes nothing else. Reasonable as a **stop-gap** while accounts are being minted; a poor destination, because its **safety** depends on everybody spelling things correctly forever.

boundary strength: convention

05

SEPARATION, SERVER-ENFORCED

One account per region — the only real wall

Give South Africa `LINEBOOKER_ZA` and Australia `LINEBOOKER_AU` . Both may now hold a stream called `ODOMETER` , because they are two namespaces. The server checks the account on every message. Nothing depends on a name being right.

DECISION

Is a region a **tenant boundary** — data that must not leak, and must not be counted twice — or only a deployment detail?

MECHANISM — MEASURED BY `LAB/03-ODOMETER.SH` , IN KILOMETRES

CLUSTER ZA · LINEBOOKER_ZA

za-1

za-2

za-3

odometer 12.5 km

account wall — server-enforced

CLUSTER AU · LINEBOOKER_AU

au-1

au-2

au-3

odometer 0 km

ACCOUNTS	ODOMETER IN ZA	ODOMETER IN AU	THE FLEET BELIEVES
one <code>LINEBOOKER</code> , both regions	25 km	25 km	50 km
<code>LINEBOOKER_ZA</code> + <code>LINEBOOKER_AU</code>	12.5 km	0 km	12.5 km

> Same subject, same code, same single publish. **Only the account changed.**

> A gateway carries interest, and it **never crosses an account**. That is the whole guarantee.

> The tenant axis in this repo is the account, so a region-per-account decision must be taken with the tenancy model open, not separately.

PROS

> **Checked by the server on every message.** Not a convention, not a filter, not a review comment.

> The obvious, domain-correct stream name works in both regions.

> Subjects stay clean — no region token, so the settled subject taxonomy survives.

> A leak becomes structurally impossible rather than unlikely.

CONS

> **Trust-chain work.** New accounts, new credentials, new exports/imports for anything that genuinely must cross.

> Cross-region sharing is now **explicit** — you must design it, per subject.

> Two write sides remain two write sides. The account makes them honest, not merged.

> **It does not help availability at all.** Card 08.

VERDICT

The destination. A count of messages can never tell you the business is wrong; a total can. 25 km for a 12.5 km trip is what pays for the migration.

boundary strength: hard

06

DURABILITY

Replicas and quorum — 1, 3, 5

A three-server cluster is **not** three copies. A stream stores the number of copies you ask for, and the default is **one**. Ask for one, lose one server, and the stream is gone.

DECISION

Per stream: how many server failures must this data survive? Every extra copy is real disk and real latency — Card 07.

MEASURED ON A 3-SERVER CLUSTER BY LAB/02-REPLICAS.SH

REPLICAS 1 — ONE COPY

za-1

za-2

za-3

→

stop it → stream is offline (10118)

REPLICAS 3 — THREE COPIES, QUORUM 2

za-1

za-2 L

za-3

→

stop the leader → new one in ~1s

REPLICAS	QUORUM	SURVIVES	STOP THE LEADER AND...
1	—	nothing	stream is offline (10118)
3	2	1 server	new leader in ~1s, messages intact
5	3	2 servers	refused — no suitable peers for placement (10005)

> Use odd numbers only — 1, 3, 5. An even count adds cost and buys no extra tolerance, because quorum is $\text{floor}(n/2)+1$. NATS caps a stream at 5.

> R5 needs five servers **in one cluster**. It will not borrow peers from the other region — a stream's replicas never span clusters (Card 03). Six servers across two clusters still gives 10005.

> Replication is **per stream**. Nothing is global, and the default is the unsafe one.

WHEN R1 IS HONESTLY RIGHT

> Data you can regenerate — a cache, a derived projection, a scratch stream.

> Development and lab work, where a rebuild costs a command.

> Very high write rates where the source of truth is elsewhere.

WHEN R1 IS A HIDDEN OUTAGE

> Anything a business decision reads. 10118 is not a slow response — it is **no response, until that one server returns**.

> Anything you cannot replay from an upstream system.

> KV buckets. A bucket is a stream, so it has replicas too — and `stream ls` hides it. Check with `nats kv ls`.

07

DURABILITY, PRICED

What replication costs — the acknowledgement

R3 does not only slow the publish. **Both kinds of acknowledgement are writes**, so both slow down: the publisher's PubAck, and the consumer's `Ack()`, because a consumer's position is replicated too.

DECISION You are sizing a throughput budget. Do you cost the extra copies as **disk**, or as **round-trips on both sides of the stream**? It is both.

MEASURED WITH `NATS BENCH`, ONE CLUSTER, SAME HARDWARE

WRITE PATH — THE PUBACK IS THE WRITE

quorum 2 → PubAck
P50 212 μs → 331 μs

ACK PATH — THE CONSUMER ACK IS ALSO A WRITE

same quorum round-trip
30.7 μs → 36.2 μs

PATH	R1	R3	COST
Synchronous publish (wait for PubAck)	4,272 msg/s · P50 212 μs	2,761 msg/s · P50 331 μs	~35% slower
Consumer fetch + Ack()	196,864 msg/s · avg 30.71 μs	121,532 msg/s · avg 36.24 μs	~38% slower

- › The consumer's `Ack()` is not free bookkeeping. It moves the consumer's position, that position lives in a replicated Raft group, so it pays the same quorum round-trip.
- › Batch and `AckAll` where the domain allows it — the fix for ack cost is fewer acks, not fewer replicas.
- › These are lab numbers on one machine. The **ratio** is the transferable part, not the rate.

THE TRAP THAT MADE THIS CARD NECESSARY

- › `nats pub` is a **core** publish. It is fire-and-forget: no PubAck, no wait, and **no error when JetStream cannot store the message**. It prints "Published" during an outage and looks like success.
- › An early quorum test here was misread for exactly this reason. The truth only appeared in the message **count**: 1 during the outage, 2 after recovery.
- › Real code must use a **JetStream publish and check the ack**. In benchmarks, use `nats bench --js` with a synchronous publish, or you are timing the network card.

VERDICT

Budget roughly **a third less throughput** for R3 on both the write and the ack path. If that is unaffordable for a stream, the honest answer is a different stream design — not R1 on data you cannot lose.

R1 → R3 ≈ -35% both ways

08

AVAILABILITY

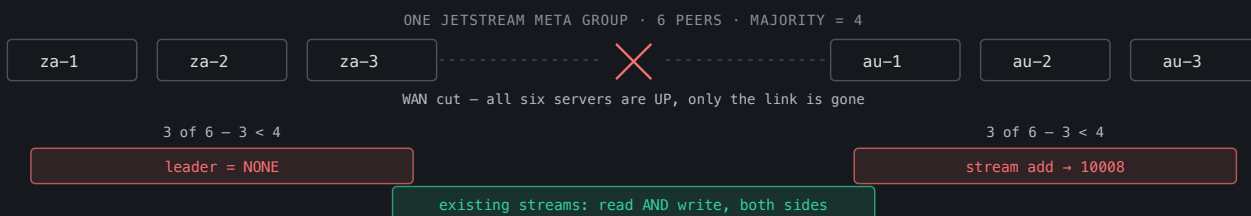
The shared failure domain — a WAN cut

A gateway makes all six servers **one JetStream meta group**. Majority of six is four. Cut the link 3 / 3 and neither side can elect a meta leader. Accounts do not change this, and neither does a domain.

DECISION

What exactly stops when the WAN goes? The honest answer is **much narrower** than "both regions are down" — but it is not nothing.

MEASURED — AU STOPPED, ZA HOLDING 3 OF 6 PEERS, **LEADER = NONE**



OPERATION ON THE ISOLATED SIDE

WORKS? MEASURED

Core NATS publish / subscribe	yes	message delivered
Read an existing stream	yes	msgs=10
Write to an existing stream	yes	10 → 15, all acked
Create / delete / edit a stream or consumer	no	JetStream system temporarily unavailable (10008)

- Existing streams keep serving because **a stream's replicas all live inside one cluster** (Card 03). That Raft group still has its own majority. Only the **meta** group is split.
- So a WAN cut is a **change freeze, not an outage**. Plan it that way: no deploys that create streams or consumers, no scaling, no new tenants — but traffic keeps flowing.
- Recovery needs no intervention. Restart the far side and the meta leader returns in seconds.

WHAT THE GATEWAY REALLY SOLD YOU

- Reach**, paid for with a **shared control plane**. The link between regions is a hard dependency for all stream management.
- The failure is **symmetric** at 3+3 — nobody wins, so there is no "primary" to fail over to.
- An account per region does not help. It is a data wall, not an availability wall.

CHECK IT IN ONE COMMAND

- `curl -s localhost:8621/jsz?meta=1 | jq '.meta_cluster'` — gives **leader** and **cluster_size**.
- leader: null** with a healthy **cluster_size** is exactly this state.
- Alert on it. It is invisible from the data plane until somebody deploys.

VERDICT

Say **"management freezes"**, never "the region is down". The difference decides whether you page somebody at 02:00 or write a ticket for the morning.

data plane: unaffected

09

AVAILABILITY, BY DESIGN

Escaping the shared failure domain

A 3 + 3 gateway is the worst arithmetic there is: an exact tie. Four shapes avoid it, and they are ordinary design choices, not tricks.

DECISION

Must **either** side keep managing streams through a cut? If yes, the topology has to change — no setting will do it.

FOUR WAYS OUT, WEAKEST FIRST		
3 + 3 — AN EXACT TIE 	3 of 6 · 3 < 4	neither side can manage streams
3 + 2 — UNEVEN, ON PURPOSE 	3 of 5 · 3 ≥ 3	the 3-side keeps its leader
3 + 3 + A HUB SITE 	hub · 4 of 7	a majority survives one lost link
SHAPE	ARITHMETIC	THROUGH A WAN CUT
Uneven regions — 3 + 2	5 peers, majority 3	the 3-side keeps its leader and stays fully writable; only the small side freezes
A third cluster (gateway hub)	3 sites, one link lost	a majority survives; the usual production answer
Nothing at all — the data plane	per-stream Raft, inside one cluster	already survives; see Card 08
Leaf nodes with their own domain	separate meta groups	genuinely independent JetStream systems — nothing to split
> No gateway at all is the fifth, degenerate case: two labs that cannot talk. Nothing collides and nothing survives a cut, because there is nothing to cut. It is not a boundary you designed. > A leaf link is the only place a JetStream domain may change (Card 01). This is what domains are actually for, and why they cannot help across a gateway.		

PREFER A HUB

> Three sites, odd count, no tie. A lost link leaves a majority.

> The hub carries no tenant data — it exists to hold a vote and to relay.

> Named as future work in [docs/Multi-Region-Plan.md](#) ; this lab has no hub.

UNEVEN SIZES ARE A REAL ANSWER

> 3 + 2 is not "one region is worse". It is a **decision about which side keeps working**, made in advance and written down.

> Say which side that is in the runbook, before the cut, not during it.

10

METHOD

Measuring without fooling yourself

Three of the conclusions in this deck were **wrong the first time**. Each was wrong for the same reason: the test tool lied, and the result looked plausible. This card is the list of traps, so they are not walked into again.

DECISION

Before you believe a distributed-systems measurement: what is the **control probe** that must still work? If it also failed, your harness is broken, not the system.

TRAP 01 · DOCKER ATE THE PORTS

- › `docker network disconnect lb-wan <container>` on a **running** container also drops that container's **published host ports**.
- › Result: 4621 and 8621 go dead and everything looks catastrophic. Reported as "both regions died". It was a Docker artefact.
- › **The giveaway:** core NATS pub/sub also "failed" — impossible from a JetStream meta split. That is the control probe.
- › **Do this instead:** `docker stop lab2-au-1 lab2-au-2 lab2-au-3`. Ports stay up, majority is genuinely lost.

TRAP 02 · "PUBLISHED" IS NOT "STORED"

- › `nats pub` is a **core** publish. No PubAck, no wait, no error when JetStream cannot store it.
- › It printed "Published" all through an outage and made a broken quorum look healthy.
- › **The giveaway:** the message **count**. 1 during the outage, 2 after recovery.
- › **Do this instead:** assert on `stream info` counts, never on a publisher's own output.

TRAP 03 · THE SHELL DID NOT SPLIT THE FLAG

- › **zsh does not word-split unquoted variables.** `C="--context lab2-za"; nats $C ls` fails with `unknown long flag '--context lab2-za'`.
- › Cascades into confusing `jq: parse error` noise that hides the real cause.
- › **Do this instead:** use an array — `C=(--context lab2-za)` — or wrap the experiment in `bash <<'EOF' ... EOF`.

THE HABITS THAT CAUGHT ALL THREE

- › **Always carry a control probe** that must survive the fault. Core pub/sub here.
- › **Assert on state, not on tool output.** Counts, `cluster.name`, `created` timestamps, files on disk.
- › **Prove the recovery too.** A test that only shows the break can be a broken test.
- › **Re-run the whole lab after every fix.** All five stages, then `lab/streams.sh` and `nats kv ls` to prove nothing was left behind.

VERDICT

A measurement you cannot explain the **mechanism** of is a rumour. Write the corrections down, dated, next to the claim they replace — this deck exists because three of them were not.

3 wrong results · all harness



CHOOSING

The whole deck, on one page

Two regions, one gateway. Three questions get asked in this order, and each has exactly one correct tool.

QUESTION 1 — DO THE TWO REGIONS COLLIDE?

TOPOLOGY	STREAMS COLLIDE?	SURVIVES A WAN CUT?
No gateway	no	yes — nothing to cut, and nothing shared
Gateway, one account	yes — 10058	management freezes on both sides
Gateway, one account per region	no	management still freezes

Q2 · WHO MAY SEE IT?

- › **The account.** Card 05.
- › Server-enforced on every message. The only wall in this deck.
- › A subject token (Card 04) is a convention and fails silently on a typo.

Q3 · WHERE DOES IT LIVE?

- › **Placement.** `--cluster za|au`. Card 03.
- › Set it explicitly, always. Replicas never span clusters.
- › `domain` is not this, and is not a boundary. Card 01.

Q4 · WHAT SURVIVES?

- › **Replicas.** R3 default, R1 by exception. Card 06.
- › Costs ~35% on publish *and* on ack. Card 07.
- › A WAN cut freezes management only. Cards 08–09.

SIX SENTENCES TO REMEMBER

- › **A gateway makes one supercluster, and a supercluster is one JetStream system with one domain.**
- › **Inside one account, a stream name is unique across the whole supercluster.**
- › **Only the account is a wall.** Subjects are conventions, placement is a location, a domain is an address.
- › **The default is Replicas 1.** Three servers is not three copies.
- › **A WAN cut is a change freeze, not an outage** — unless the regions are unevenly sized, in which case only the smaller one freezes.
- › **A business total finds bugs a message count cannot.** 25 km for a 12.5 km trip.

VERDICT

Gateway plus one account per region plus explicit placement plus R3. Then decide, in advance and in writing, which side keeps managing streams when the link drops.

10 cards · 1 default shape

Where every number came from

This deck is a lab record, not a literature review. Every figure was printed by a script in this repository, on six NATS 2.14 servers, in September 2026.

CARDS 01-02 · NAMESPACE

- > `demos/02-multi-region/lab/00-the-problem.sh` — the 10058 collision, and AU reading ZA's stream
- > `demos/02-multi-region/nats/nats.conf` — the single `lb` domain, with the story
- > `demos/02-multi-region/docs/Multi-Region-Plan.md` D9 / D11 — the dated correction

CARDS 03-05 · PLACEMENT, SUBJECTS, ACCOUNTS

- > `lab/01-option-1-subjects.sh` — the region token, and what it costs
- > `lab/01-the-wall.sh` — does an account hold across a gateway?
- > `lab/03-odometer.sh` + `odometer/` — the 12.5 km / 25 km measurement
- > `lab/streams.sh` — every stream, every account, which cluster it sits on

CARDS 06-07 · REPLICAS AND ACK COST

- > `lab/02-replicas.sh` — R1 offline (10118), R3 re-elects, both passes on one stream name
- > `nats bench` , JetStream mode, synchronous publish and consumer fetch/ack
- > 10005 `no suitable peers` — R5 on three servers, and R5 across the supercluster

CARDS 08-10 · WAN CUT AND METHOD

- > Clean fault injection: `docker stop lab2-au-1 lab2-au-2 lab2-au-3`
- > `curl -s localhost:8621/jsz?meta=1` — `meta_cluster.leader` , `cluster_size`
- > `demos/02-multi-region/deploy/compose.za.yaml` — the dated warning against `docker network disconnect`
- > `demos/02-multi-region/diagrams/gateway-vs-wan-cut.html` — the same finding, drawn

COMPANION DOCUMENTS

- > `demos/02-multi-region/README.md` — the lab intro, three questions
- > `demos/02-multi-region/CLAUDE.md` § "Measured facts — do not re-guess these"
- > `diagrams/gateway-double-capture-options.html` — the option comparison, drawn